

Compilation and Repair Loops

This section closes the loop between generated prose and the compiled paper. Once section agents have written their payloads, the repository still needs a document-level assembly step that turns many isolated `.tex` fragments into a single buildable manuscript. In practice, that means the system must concatenate or `cat` the section payloads in outline order, resolve shared macros and package dependencies, and run the LaTeX toolchain against the assembled document until the build succeeds or produces actionable diagnostics.

The build log is not a disposable byproduct. It is a primary coordination surface for repair. A successful compile confirms that the manuscript is structurally consistent, while a failed compile identifies the exact file, line, and token context where the document broke. Because section payloads are owned by individual section agents, the repair loop should preserve that ownership boundary: the agent responsible for a section should receive the relevant diagnostics and make the smallest plausible correction to its own payload rather than rewriting unrelated material.

This minimal-repair discipline matters for two reasons. First, it keeps the provenance of changes legible: a compile error in one section should not trigger broad, speculative edits elsewhere in the book. Second, it reduces the risk of introducing new defects while fixing the old one. A section agent can usually respond to a diagnostic by correcting a missing brace, repairing an invalid citation command, adjusting an environment boundary, or rephrasing a sentence that overflows a fragile construct. The goal is not to optimize the prose in the same pass as the repair; the goal is to restore a clean build with the smallest safe change.

Compile diagnostics also support section attribution. When the assembler reports an error, the system can map the failing line back to the owning section payload and then to the corresponding section agent. That attribution is important in a multi-agent manuscript because the compiled document is a shared artifact, but the repair responsibility is local. The build log therefore functions as a routing mechanism: it tells the hypervisor or gardener which section agent should be asked to revise, and it gives that agent enough context to act without guessing.

Not every build issue is a syntax error. Some failures are editorial or design problems that only become visible after the manuscript is rendered. A page break may be awkward, a figure may be too large, a paragraph may create an undesirable widow or orphan, or a section may read correctly in isolation but feel unbalanced in the compiled layout. These are document-design feedback signals rather than pure compilation faults. They belong to the same repair loop, but they are routed differently: instead of asking for a mechanical fix to the TeX source, the system may ask for a local rewrite, a figure adjustment, or a layout-aware revision that improves the final page composition.

The important design choice is that compilation is not the end of authoring; it is a feedback stage. The manuscript is assembled, compiled, inspected, and then repaired in a bounded loop until the output is both syntactically valid and editorially acceptable. In that sense, the build log is part of the manuscript’s memory. It records where the system failed, which section owned the failure, what kind of repair was needed, and whether the resulting document satisfied both LaTeX and design constraints. That durable feedback loop is what turns a collection of section drafts into a reliable compiled paper.

A practical repair loop therefore has three steps. First, assemble the manuscript from the current section payloads and run the compiler. Second, inspect the log and classify the failure as a local syntax problem, a missing dependency, or a document-design issue. Third, route the smallest appropriate task back to the owning section agent or to the document-level design workflow. This keeps the system from treating compilation as a monolithic pass and instead makes it a controlled exchange between build output and targeted revision.

In the broader chapter, this section completes the artifact lifecycle: source material becomes

section payloads, section payloads become reviewable diffs, and reviewable diffs become compiled outputs that can still fail in ways worth repairing. The result is not merely a buildable manuscript, but a manuscript whose build history is itself inspectable and actionable.